Ensembles and Multi-Output Regression

Advanced Machine Learning

Sam Ogden

DRAFT

This slide deck is still under active development. Expect changes before it is finalized.

Learning objectives

- Define an ensemble accurately: several models, the *same* target, combined into one better prediction — with no requirement that training be independent or that combination be a fixed rule
- Recognize the same “several models” idea reused for a different question: predicting several *different* things at once
- Extend linear regression to multiple *different* outputs by stacking weights into a matrix W and biases into a vector b

One question, or several?

So far: one input, one number out

- Last lecture we built a model to predict a single number
  - **What happens when we want more than one number out?**

- There are two ways to phrase this question — and they're our two sections today
  - More than one **guess** at the *same* thing (e.g. several predictors, one house price)
  - More than one **different** thing to predict (e.g. tomorrow's high *and* low)

Ensembles

Predicting one thing, many models

The Big Idea

A bunch of so-so predictors, combined the right way, can beat any one of them alone.

Several predictors, one question

- Predict **one house's price** from a couple of noisy features
  - Instead of one model, train *several* simple predictors at the same target
- Each one, alone, is kind of mediocre
  - Together they can be excellent predictors

This is an ensemble

- Several models
- All aimed at the **same** target
- Combined into **one** better prediction

> **Tip**
>
> Notice what's *not* in this definition — nothing about how the models are trained, and nothing about how they're combined. Both of those turn out to vary.

How to go about making an ensemble model

Where do we get different predictors

- Ensemble models vary:
  - Train data subsampling
  - Different architectures
  - Initialization
  - **They must vary something**

Examples of ensemble methods:

- Random Forests
  - Make a bunch of little yes/no decisions
- Bagging
  - We subsample training data
- Boosting
  - Train each model to correct for the error of the previous model

We'll be looking at a toy version, but the same idea holds

How do they get combined?

Fixed rule

- Decided ahead of time
- Averaging, voting, weighted averaging

Learned rule: stacking

- Train a small model whose job is to learn how to weigh the base predictions
- Still just $Wx + b$ — on the base predictions as x
  - The only difference is the inputs are the outputs of each predictor

Worked example: stacking the house-price ensemble

Say we have three models:

1. A: KNN – pick the three closest houses
2. B: Zip Code Average
3. C: Linear Model (like last lecture)

We could combine them in a few ways

Averaging:

- We simply say $\frac{A+B+C}{3}$

Stacking:

- We might say $0.5 * A + 0.1 * B + 0.4 * C$
  - But these would be *learned weights*

How do the base models get trained?

- Usually **independent** — trained separately, no shared parameters (bagging, random forests, our toy example so far)
  - Not required, though

> **Note**
>
> **Boosting** trains each model *sequentially* — every new model is trained to correct the error of the previous one. That's not independent training at all, and it's still unambiguously called an ensemble method. Real terminology has blurry edges like this.

Multi-Output Models

Predicting multiple things

Parallel models, different predictions

- Same pattern as ensembles: **several models, trained side by side**, all fed the same input
- Difference: instead of combining their outputs into **one** answer, each model produces its **own**, different output
- So instead of “combine N guesses into 1,” we now have “1 input → N different outputs”

The Naive Approach

- Predict tomorrow's **high** *and* **low** temperature from today's readings
- We train two models, feeding measurements into both
  - One for **high** and one for **low**
  - Different w, different b, trained independently — neither model knows the other exists

Worked example: two separate models

Today's readings: high $= 75^\circ$, low $= 55^\circ$, humidity $= 40\%$ → $x = [75, 55, 40]$

	weights w	**bias** b	**prediction for today's** x
High-temp model	$[0.8,\ 0.1,\ -0.05]$	10	$0.8(75) + 0.1(55) - 0.05(40) + 10 = 73.5$
Low-temp model	$[0.3,\ 0.6,\ -0.02]$	5	$0.3(75) + 0.6(55) - 0.02(40) + 5 = 59.7$

- Nothing shared — different weights, different bias, computed completely independently

Contrast with the ensemble

The ensemble (last section)

- A, B, C — three predictors, same target: house price
- No shared parameters between them
- Combined into **one** final price

Here

- High-model, low-model — two predictors, *different* targets
- No shared parameters between them, either
- Kept as **two** separate answers, never combined

> **Tip**
>
> Same skeleton both times — independent models, no shared parameters. The only thing that changed is what question each model is answering.

Two models is two of everything

- Two weight vectors, two biases, two training loops to manage — even though both models are doing the exact same *kind* of computation, on the exact same input
- Nothing wrong with the predictions themselves — this is purely about **not wanting to juggle K separate model objects**

> What if, instead of two separate outputs in two separate models, the output were just a single **vector**?

The Big Idea

Give each *output* its own row of weights, but they all see the same input.

From a vector of weights to a matrix

- Same dot product idea as before, just repeated per output, and now trained together:

$$\hat{\mathbf{y}} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

- $\mathbf{W}$ has one row per output, $\mathbf{b}$ one entry per output

Worked example: the same weights, now one matrix

$$\underbrace{\begin{bmatrix} 0.8 & 0.1 & -0.05 \\ 0.3 & 0.6 & -0.02 \end{bmatrix}}_{W,\ 2\times 3} \underbrace{\begin{bmatrix} 75 \\ 55 \\ 40 \end{bmatrix}}_{x,\ 3\times 1} + \underbrace{\begin{bmatrix} 10 \\ 5 \end{bmatrix}}_{b,\ 2\times 1} = \underbrace{\begin{bmatrix} 73.5 \\ 59.7 \end{bmatrix}}_{\hat{y},\ 2\times 1}$$

- Row 1 of W is exactly the “high-temp model” from before; row 2 is exactly the “low-temp model”
- Same numbers as our two separate models — just stacked into one W

Breakdown: one row of W

- Row j of W, dotted with x, plus b_j — that's exactly single-output regression from last lecture
- This row has nothing to do with the other rows — it's the *same input* x, seen K times, once per output

Worked example: pulling out just row 1

$$\underbrace{[0.8,\ 0.1,\ -0.05]}_{w_1,\ \text{row 1 of } W} \cdot \underbrace{[75,\ 55,\ 40]}_{x} + \underbrace{10}_{b_1} = 73.5 = \hat{y}_1$$

- Same computation, same numbers, as the “high-temp model” a few slides ago — row 1 doesn’t know row 2 exists

Breakdown: what stacking rows means

- K outputs → K rows → W is $K \times (\text{number of features})$
- Every row reads the same x; each row has different weights!
  - So we now have K models for K outputs

The loss, extended

The loss is exactly the same, it's just summed over all K of the output predictions

- Still one scalar loss number — sum (or average) per-output MSE across all outputs:

$$L = \sum_{j=1}^{K} (\hat{y}_j - y_j)^2$$

Breakdown: two sums, not one

Once we have a whole batch of N examples, the full loss has **two nested sums**:

$$L = \frac{1}{N} \sum_{i=1}^{N} \underbrace{\sum_{j=1}^{K} \left(\hat{y}_j^{(i)} - y_j^{(i)} \right)^2}_{\text{one example's error, all } K \text{ outputs}}$$

- **Inner sum** — over the K outputs, for *one* example (this is the "loss, extended" formula from the last slide)
- **Outer sum** — over the N examples in the batch, exactly like every loss we've averaged over so far
- Easy to blur these together since they're both just "$\sum$" — they're summing over two completely different things

The gradient, extended

- Same “prediction minus truth, times input” shape as last lecture — once per output row
- Side by side with last lectures’s single-output gradient: nothing new *per row*, just more rows

Worked example: gradients for row 1 and row 2

Say the true tomorrow was $y = [78,\ 55]$, so our predictions ($\hat{y} = [73.5,\ 59.7]$) were off by $e_1 = -4.5$ and $e_2 = 4.7$:

$$\frac{\partial L}{\partial w_1} = 2e_1\, x = 2(-4.5)[75, 55, 40], \qquad \frac{\partial L}{\partial b_1} = 2e_1 = -9$$

$$\frac{\partial L}{\partial w_2} = 2e_2\, x = 2(4.7)[75, 55, 40], \qquad \frac{\partial L}{\partial b_2} = 2e_2 = 9.4$$

- $\partial L / \partial w_1$ only involves e_1 — row 2's error never shows up in row 1's gradient, and vice versa

SGD is the exact same as it was before

- Pick a mini-batch, compute predictions $\hat{\mathbf{y}} = Wx + b$ for each example in it
- Compute the gradient of the loss with respect to W and b — same formulas as the last slide, once per row
- Update: $W \leftarrow W - \eta \nabla_W L$, and $b \leftarrow b - \eta \nabla_b L$
- Repeat for the next mini-batch — same loop as last lecture

> **Note**
>
> Nothing about the training loop changes — W is just matrix-shaped now.

Back to our example: predicting high and low temperatures

$$W = \begin{bmatrix} 0.8 & 0.1 & -0.05 \\ 0.3 & 0.6 & -0.02 \end{bmatrix}, \quad b = \begin{bmatrix} 10 \\ 5 \end{bmatrix}, \quad x = \begin{bmatrix} 75 \\ 55 \\ 40 \end{bmatrix}$$

- Same running example, same numbers — but now **one** W/b pair instead of two separate models
- The rows still don't talk to each other during training — same independent gradients as before — we've just packaged them into a single object instead of K separate ones
- That packaging is what lecture 05 needs: one matrix multiply producing a whole vector of scores, ready for softmax to act on

Summary

The recipe

	same or different targets?	how combined / shared
Ensemble — fixed rule	same	averaging / voting
Ensemble — stacking	same	learned
No-sharing baseline	different	none — separate outputs
Multi-output regression	different	shared W, trained jointly

> **Note**
>
> "The model outputs a vector" is now old news — lecture 05 picks this up immediately for classification, using the **multi-output regression** row's machinery specifically.

> **Tip**
>
> If stacking was cut here: it's coming back in lab.